

Day 5: Effects, Refs and Custom Hooks

Complete Lab Steps and Code

This guide builds on the interactive TaskBoard from Day 4 Lab 4.3. You will add debounced product search, a weather dashboard powered by a reusable fetch Hook, then browser storage, API seed data, focus management and a live tab title.

Each checkpoint builds on the previous one. Lab 5.3 is the final Day 5 app: the temporary search and weather panels are removed, and TaskBoard persists its tasks.

Before you start

Requirements: Node.js 24 LTS, npm, and an internet connection for the public API calls.

Start from `Projects/Day05_Effects_Refs_and_Custom_Hooks/starter/taskboard/`, or continue with your own Day 4 Lab 4.3 TaskBoard.

```
cd Projects/Day05_Effects_Refs_and_Custom_Hooks/starter/taskboard
npm install
npm run dev
```

Open the Local URL printed by Vite. Run these checks at each checkpoint:

```
npm run lint
npm run build
```

The Day 5 starter already includes the complete Day 4 interactive TaskBoard and the catalogue-related files. Do not replace its existing task CRUD logic while adding the temporary lab components.

Lab 5.1: Search-as-You-Type with Debouncing

Goal: Search DummyJSON products without sending a request for every keystroke.

Steps

1. Create `src/components/ProductSearch.jsx` with a controlled text input.
2. Keep both the immediate query and delayed `debouncedQuery` in state.
3. Add an effect that schedules the delayed query 400 ms after typing; clear the previous timer in cleanup.
4. Add a second effect that fetches DummyJSON using the delayed query. Encode it with `encodeURIComponent`.
5. Skip search requests for queries shorter than two characters.
6. Cancel obsolete fetches with `AbortController` and ignore abort errors.
7. Render the too-short, loading, error, empty-results and results states.
8. Import `ProductSearch` into `App.jsx` and render it below the board in a temporary practice section.
9. Check the browser Network tab: typing quickly should result in a single request after the pause. Run lint/build and commit this checkpoint if you use version control.

`src/components/ProductSearch.jsx`

```
import { useEffect, useState } from "react";

function ProductSearch() {
  const [query, setQuery] = useState("");
  const [debouncedQuery, setDebouncedQuery] = useState("");
  const [result, setResult] = useState({
    query: "",
    products: [],
    error: null,
  });

  useEffect(() => {
    const timeoutId = setTimeout(() => setDebouncedQuery(query), 400);
    return () => clearTimeout(timeoutId);
  }, [query]);

  const tooShort = debouncedQuery.trim().length < 2;

  useEffect(() => {
    if (tooShort) return;

    const controller = new AbortController();
    const url = `https://dummyjson.com/products/search?q=${encodeURIComponent(debouncedQuery)}`;

    fetch(url, { signal: controller.signal })
      .then((response) => {
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        return response.json();
      })
      .then((json) => {
        setResult({
          query: debouncedQuery,
          products: json.products,
          error: null,
        });
      })
      .catch((error) => {
        if (error.name !== "AbortError") {

```

```

        setResult({
          query: debouncedQuery,
          products: [],
          error: error.message,
        });
      }
    });

    return () => controller.abort();
  }, [debouncedQuery, tooShort]);

const loading = !tooShort && result.query !== debouncedQuery;

let content;
if (tooShort) {
  content = <p>Type at least 2 characters.</p>;
} else if (loading) {
  content = <p>Searching...</p>;
} else if (result.error) {
  content = <p role="alert">Search failed: {result.error}</p>;
} else if (result.products.length === 0) {
  content = <p>No products found.</p>;
} else {
  content = (
    <ul>
      {result.products.map((product) => (
        <li key={product.id}>
          {product.title}: ${product.price}
        </li>
      ))}
    </ul>
  );
}

return (
  <section>
    <label htmlFor="search">Search products</label>
    <input
      id="search"
      value={query}
      onChange={(event) => setQuery(event.target.value)}
    />
    {content}
  </section>
);
}

export default ProductSearch;

```

Loading is derived from whether the stored result belongs to the current delayed query. This avoids setting loading state synchronously at the start of an effect.

Update src/App.jsx for Lab 5.1

Keep the existing Day 4 task state and handlers. Add this import:

```
import ProductSearch from "../components/ProductSearch.jsx";
```

Then add this after the existing Board in the returned fragment:

```

<section className="practice">
  <h2>Lab 5.1 product search</h2>
  <ProductSearch />
</section>

```

Append to src/App.css for Lab 5.1

```
.practice label {  
  margin-right: 8px;  
}  
  
.practice input,  
.practice select {  
  font: inherit;  
  padding: 4px 8px;  
}
```

Done when: typing quickly sends one request, old requests are cancelled, and all search states display correctly. If DummyJSON is unavailable on your network, the handbook suggests using JSONPlaceholder users and filtering by name instead.

Lab 5.2: Weather Dashboard from a Public API

Goal: Create a reusable useFetch Hook and use it to load Open-Meteo weather for a selected city.

Open-Meteo needs no API key. The URL uses latitude, longitude, current temperature, current wind speed and the selected location's timezone.

Steps

1. Create `src/hooks/useFetch.js`. It accepts a URL and returns `{ data, error, loading }`.
2. Fetch in an effect, check `response.ok`, parse JSON, and cancel the request during cleanup with `AbortController`.
3. Set `loading` to `false` only when the active request finishes; an aborted request must not update state.
4. Create `src/components/WeatherDashboard.jsx` with a select for Johannesburg, Cape Town and Durban.
5. Derive the selected city's coordinates from the selected city name; build the URL as a string.
6. Show `loading`, `error`, temperature, wind speed and last-updated time.
7. In `App.jsx`, remove the `ProductSearch` import/panel and render `WeatherDashboard` in the practice section instead.
8. Change the city and verify the request changes in the Network tab. Run lint/build and commit this checkpoint if you use version control.

`src/hooks/useFetch.js`

```
import { useEffect, useState } from "react";

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const controller = new AbortController();

    fetch(url, { signal: controller.signal })
      .then((response) => {
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        return response.json();
      })
      .then((json) => {
        setData(json);
        setError(null);
      })
      .catch((fetchError) => {
        if (fetchError.name !== "AbortError") {
          setError(fetchError.message);
        }
      })
      .finally(() => {
        if (!controller.signal.aborted) setLoading(false);
      });

    return () => {
      controller.abort();
      setLoading(true);
    };
  }, [url]);
}
```

```
    return { data, error, loading };
  }
}
```

src/components/WeatherDashboard.jsx

```
import { useState } from "react";
import { useFetch } from "../hooks/useFetch.js";

const cities = [
  { name: "Johannesburg", lat: -26.2, lon: 28.05 },
  { name: "Cape Town", lat: -33.92, lon: 18.42 },
  { name: "Durban", lat: -29.86, lon: 31.02 },
];

function WeatherDashboard() {
  const [cityName, setCityName] = useState(cities[0].name);
  const city = cities.find((item) => item.name === cityName);

  const url =
    "https://api.open-meteo.com/v1/forecast" +
    `?latitude=${city.lat}&longitude=${city.lon}` +
    "&current=temperature_2m,wind_speed_10m&timezone=auto";
  const { data, loading, error } = useFetch(url);

  let content;
  if (loading) {
    content = <p>Loading weather...</p>;
  } else if (error) {
    content = <p role="alert">Could not load the weather: {error}</p>;
  } else {
    content = (
      <div>
        <p>Temperature: {data.current.temperature_2m} Â°C</p>
        <p>Wind speed: {data.current.wind_speed_10m} km/h</p>
        <p>Last updated: {data.current.time.replace("T", " ")}</p>
      </div>
    );
  }

  return (
    <section>
      <h2>Weather in {city.name}</h2>
      <label htmlFor="city">City </label>
      <select
        id="city"
        value={cityName}
        onChange={(event) => setCityName(event.target.value)}
      >
        {cities.map((item) => (
          <option key={item.name} value={item.name}>
            {item.name}
          </option>
        ))}
      </select>
      {content}
    </section>
  );
}

export default WeatherDashboard;
```

Update src/App.jsx for Lab 5.2

Remove the ProductSearch import and its section. Add this import:

```
import WeatherDashboard from "../components/WeatherDashboard.jsx";
```

Render the dashboard after the board:

```
<section className="practice">  
  <WeatherDashboard />  
</section>
```

The URL is a plain string, so it changes only when the selected city's coordinates change. useFetch is shared logic: the dashboard does not implement its own fetch effect.

Done when: changing the city updates its weather, the old request is cancelled, and the console has no errors. If data is null, show the loading/error state before reading data.current.

Lab 5.3: TaskBoard Persistence and Seed Data

Goal: Keep tasks after refresh and load five initial tasks from JSONPlaceholder on first use.

Steps

1. Create `src/hooks/useLocalStorage.js`. It should provide the same value/setter pair as `useState`, reading initial data from `localStorage` and saving changes as JSON.
2. In `App.jsx`, replace `task` `useState` with `useLocalStorage("tasks", null)`. `null` means there is no saved board yet; an empty array is a valid saved board.
3. Derive `needsSeed` from `tasks === null`. Fetch only when it is true.
4. Cancel the seed request in effect cleanup. Map API todos into TaskBoard tasks with string IDs, status and points.
5. Render a loading message while the `tasks` value is still null.
6. Keep the Day 4 add, move, rename and delete handlers, changing only their state setter to `setTasks` from the custom Hook.
7. Add a Reset board button that sets `tasks` to null, causing the seed effect to run again.
8. Add a ref to the title input in `AddTaskForm` and focus it on mount.
9. Update the browser tab title from the open task count in `Header`.
10. Test add/edit/move/delete, refresh persistence and reset. Run lint/build and commit/push if you use version control.

Do not check `localStorage` from the seeding effect. The custom Hook handles storage; `tasks === null` is the signal that seeding is needed. Also, keep every Hook call above the loading early return.

`src/hooks/useLocalStorage.js`

```
import { useEffect, useState } from "react";

export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored !== null ? JSON.parse(stored) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}
```

Final `src/App.jsx`

This complete file replaces the Day 4 App. The Day 4 task handlers remain, but search and weather are no longer mounted in the final app.

```
import { useEffect } from "react";
import "./App.css";
import Header from "../components/Header.jsx";
import AddTaskForm from "../components/AddTaskForm.jsx";
import Board from "../components/Board.jsx";
import { useLocalStorage } from "../hooks/useLocalStorage.js";
```

```

const SEED_URL = "https://jsonplaceholder.typicode.com/todos?_limit=5";

function App() {
  const [tasks, setTasks] = useLocalStorage("tasks", null);
  const needsSeed = tasks === null;

  useEffect(() => {
    if (!needsSeed) return;

    const controller = new AbortController();
    fetch(SEED_URL, { signal: controller.signal })
      .then((response) => {
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        return response.json();
      })
      .then((todos) => {
        setTasks(
          todos.map((todo) => ({
            id: String(todo.id),
            title: todo.title,
            status: todo.completed ? "done" : "todo",
            points: 1,
          }))
        )
      })
      .catch((error) => {
        if (error.name !== "AbortError") console.error(error);
      });

    return () => controller.abort();
  }, [needsSeed, setTasks]);

  function handleAdd(newTask) {
    const id = crypto.randomUUID();
    setTasks((previousTasks) => [
      ...previousTasks,
      { ...newTask, id, status: "todo" },
    ]);
  }

  function handleStatusChange(id, status) {
    setTasks((previousTasks) =>
      previousTasks.map((task) =>
        task.id === id ? { ...task, status } : task
      )
    );
  }

  function handleRename(id, title) {
    setTasks((previousTasks) =>
      previousTasks.map((task) =>
        task.id === id ? { ...task, title } : task
      )
    );
  }

  function handleDelete(id) {
    setTasks((previousTasks) =>
      previousTasks.filter((task) => task.id !== id)
    );
  }

  if (tasks === null) return <p>Loading starter tasks...</p>;

  return (
    <>
      <Header tasks={tasks} />
      <button className="reset-button" onClick={() => setTasks(null)}>

```

```

        Reset board
      </button>
      <AddTaskForm onAdd={handleAdd} />
      <Board
        tasks={tasks}
        onStatusChange={handleStatusChange}
        onRename={handleRename}
        onDelete={handleDelete}
      />
    </>
  );
}

export default App;

```

When reset is clicked, setting tasks to null is saved by the custom Hook. On the next render, needsSeed becomes true and the effect fetches the seed data again. To make an empty board instead, set tasks to [].

Final src/components/Header.jsx

```

import { useEffect } from "react";

function Header({ tasks }) {
  const doneCount = tasks.filter((task) => task.status === "done").length;
  const openCount = tasks.filter((task) => task.status !== "done").length;

  useEffect(() => {
    document.title = `TaskBoard (${openCount} open)`;
  }, [openCount]);

  return (
    <header className="header">
      <h1>TaskBoard</h1>
      <p>{doneCount} of {tasks.length} done</p>
    </header>
  );
}

export default Header;

```

Final src/components/AddTaskForm.jsx

```

import { useEffect, useRef, useState } from "react";

const emptyForm = { title: "", assignee: "", points: 1 };

function AddTaskForm({ onAdd }) {
  const [form, setForm] = useState(emptyForm);
  const [error, setError] = useState("");
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus();
  }, []);

  function handleChange(event) {
    const { name, value } = event.target;
    setForm((previousForm) => ({ ...previousForm, [name]: value }));
  }

  function handleSubmit(event) {
    event.preventDefault();
    const title = form.title.trim();

```

```

    if (title.length < 3) {
      setError("Title needs 3+ characters.");
      return;
    }

    setError("");
    onAdd({
      title,
      assignee: form.assignee.trim(),
      points: Number(form.points),
    });
    setForm(emptyForm);
  }

  return (
    <form className="add-task-form" onSubmit={handleSubmit}>
      <label htmlFor="title">Title</label>
      <input
        id="title"
        name="title"
        ref={inputRef}
        value={form.title}
        aria-invalid={Boolean(error)}
        onChange={handleChange}
      />

      <label htmlFor="assignee">Assignee</label>
      <input
        id="assignee"
        name="assignee"
        value={form.assignee}
        onChange={handleChange}
      />

      <label htmlFor="points">Points</label>
      <input
        id="points"
        name="points"
        type="number"
        min="1"
        value={form.points}
        onChange={handleChange}
      />

      {error && <p role="alert">{error}</p>}
      <button type="submit">Add task</button>
    </form>
  );
}

export default AddTaskForm;

```

Append to src/App.css for Lab 5.3

The starter already includes earlier Day 4 styles. Add this rule for the reset button:

```

.reset-button {
  font: inherit;
  margin-bottom: 16px;
}

```

Done when: tasks survive refresh, seed data loads only when no tasks are stored, and the browser tab title shows the current open count.

Final Check

From the TaskBoard project folder:

```
npm run lint  
npm run build
```

Then verify:

- Search waits for a 400 ms pause and stale requests are cancelled.
- Switching cities loads weather for the selected city.
- TaskBoard loads seed data the first time, and the title input receives focus.
- Adding, moving, renaming and deleting tasks still works.
- Refreshing preserves tasks. Reset board loads the seed tasks again.
- The browser tab title changes as tasks are completed or reopened.

The finished Lab 5.3 checkpoint is the Day 6 starting point. Day 6 converts it to TypeScript and adds routing.